Sport Assistance

ANSWER KEY (full solutions). Build a personalized AI sports coach in Python + Gradio

Student tutorial: follow the steps in order and build the project as you read.

Contents

Welcome: what you are building **3**

How to use this tutorial 3

The roadmap 3

Before you start **5**

Set up your project 5

Demo mode (no API key needed) 6

The big picture 6

Build it, step by step **7**

Step 1: Run the starter and study the worked example 7

Step 2: Feature 2, create the athlete profile 9

Step 3: Feature 1, coaching topics for each sport 12

Step 4: Feature 3, describe your playing style 14

Step 5: Feature 5, the personalized system prompt 16

Step 6: Feature 6, trace the chat, then talk to your coach 19

Step 7: Feature 4, the AI playing-style analysis prompt 23

Step 8: Feature 7, update the profile and confirm the summary 26

Step 9: Optional extensions 29

What you learned 29

Welcome: what you are building

Sport Assistance is a chat app with an AI coach that coaches **you**, not "an athlete in general". You fill in a profile (your sport, experience, dominant hand, strengths, weaknesses, preferences) and describe how you play. Then you chat with a coach that remembers all of it. By the end you will be able to:

1. Pick a sport and see coaching topics that fit it.
2. Save an **athlete profile** and update it at any time.
3. Describe your playing style and get an AI summary you can confirm or correct.
4. Chat with a coach whose answers change depending on your profile (for example, a left-handed beginner gets different advice from a right-handed advanced player).

The folder you were given is a **scaffold**. The app already launches and runs in demo mode, but the important functions are empty and marked `TODO`. This tutorial walks you through every `TODO`, in the order you should do them. For each one you will find:

- **Goal**: what the function should do, in plain English.
- **Ideas you need**: the small bit of Python being practiced.
- **Do it**: step-by-step instructions with code.
- **Check it**: how to run it and what you should see.

> **TIP**
> Don't just copy and paste. Type the code yourself, and after each step run the app. The fastest way to learn is to break something, read the error, and fix it.

How to use this tutorial

- Keep this PDF open next to your editor. Work through the steps **in order**. Each step builds on the last.
- Code shown with a file name above it (for example `profile.py`) belongs in that file.
- Whenever you see **Checkpoint**, stop and test before moving on.
- If you get stuck, read the **Stuck?** boxes. They list the mistakes students make most often.

The roadmap

Step	File	What you build
1	`app.py`	Run the starter and study the worked example, `handle_save_profile`
2	`profile.py`	Feature 2: create the profile dictionary
3	`coach.py`	Feature 1: coaching topics for each sport
4	`profile.py`	Feature 3: store the playing-style description
5	`coach.py`	Feature 5: the personalized **system prompt** (the heart of the project)
6	`app.py`, `coach.py`, `ai_helper.py`	Feature 6: trace the chat, then try it with a real key

Step	File	What you build
7	`coach.py`	Feature 4: the AI playing-style analysis prompt
8	`profile.py` , `app.py`	Feature 7: update the profile, and confirm the style summary
9	any	Optional extensions

Before you start

Set up your project

Open a terminal inside the `sport_assistance` folder, then run the commands for your computer **one at a time**.

Open the terminal in the right folder:

- **Mac:** open the **Terminal** app, type `cd ` (with a space), drag the `sport_assistance` folder onto the window, and press Enter. (Or right-click the folder in Finder and choose **New Terminal at Folder**.)
- **Windows:** open the `sport_assistance` folder in File Explorer, click the address bar at the top, type `powershell`, and press Enter. A PowerShell window opens already inside the folder.
- **Either:** if you use VS Code, open the folder and choose **Terminal > New Terminal**.

On a Mac:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python app.py
```

On Windows (PowerShell or Command Prompt):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python app.py
```

What each line does:

- `python -m venv .venv` creates a private "box" of Python packages just for this project. (On a Mac the command is `python3` because many Macs have no plain `python` until the box is on.)
- `source .venv/bin/activate` (Mac) or `.venv\Scripts\activate` (Windows) steps into that box. You should now see `(.venv)` at the start of your terminal line. **Do this every time you open a new terminal.**
- `pip install -r requirements.txt` installs Gradio (the web page), the Google GenAI library (the AI) and python-dotenv (reads your key).
- `cp .env.example .env` (Mac) or `copy .env.example .env` (Windows) makes your private settings file. You can leave the key blank for now.
- `python app.py` starts the app. Open the address it prints (usually `http://127.0.0.1:7860`).

> **STUCK?**
> **Windows:** if `python` is "not found", try `py -m venv .venv` instead (then use plain `python` once the box is on). If PowerShell says *running scripts is disabled on this system* when you activate, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer **Y**, and try again. Or just use **Command Prompt** instead of PowerShell. **Mac:** if `python3` is not found, install Python from python.org.

To stop the app, click the terminal and press `Ctrl + C`. **After you edit a Python file you must stop and restart the app** to see your change.

Demo mode (no API key needed)

The AI parts need a Google Gemini API key, which your teacher may give you later (or you can create one at Google AI Studio, aistudio.google.com/apikey). **Without a key the app runs in demo mode.** The coach gives a clearly labelled canned reply instead of real advice. You can finish Steps 1 to 5 and test everything by printing prompts, so you do not need a key until Step 6.

> **WARNING**
> Never paste your API key into a Python file, share it, or commit `.env` to GitHub. Keys go only in the `.env` file. (The project's `.gitignore` already ignores it.)

The big picture

The whole project is one idea. The AI knows **nothing** about you, so our code writes everything it needs to know into a block of text called a **prompt**:

```
form inputs ---> profile dict ---> prompt string ---> LLM ---> personalized advice
(Gradio)         (profile.py)      (coach.py)       (ai_helper.py)   (shown in Gradio)
```

1. **Profile.** The form values become a Python dictionary (`profile.py`). Gradio keeps it in a `gr.State` so it is remembered while the page is open.
2. **Prompt.** `coach.py` turns the dictionary into text: "You are a basketball coach. The athlete is left-handed, a beginner..."
3. **LLM call.** `ai_helper.py` sends that text plus the conversation to the AI.
4. **Response.** The reply appears in the chat window.

File	Job
`app.py`	The screen: the form, the tabs, the chat, and the functions that run when you click
`profile.py`	Creating and updating the profile dictionary (Features 2, 3, 7)
`coach.py`	Topics, prompt building, and demo replies (Features 1, 3, 4, 5, 6)
`ai_helper.py`	The only file that talks to the AI. **You never need to edit it.**

> **NOTE**
> `profile.py` has the same name as an old, rarely used Python library module. That is harmless here, because Python looks in your project folder first. Just do not run `python -m cProfile` in this folder.

Build it, step by step

Step 1: Run the starter and study the worked example

Goal: understand one complete, finished handler before you write your own. There is nothing to type in this step.

1a. Click around

Run `python app.py`. There are four tabs:

1. **Profile**: fill in the form and click **Save profile**. You will see a plain summary. Notice the preferences and strengths come out as `not provided`, and handedness is always `N/A`. That is because `create_profile` is a stub. Fixing it is your first job.
2. **Playing style**: two buttons that will use your profile. Right now **Confirm summary** says it is not saved.
3. **Chat with your coach**: type a message. You get a canned demo reply.
4. **Update profile**: choose a field and a new value. Nothing changes yet.

Also notice that when you change the **Sport** dropdown, the **Coaching topics** list barely changes. That is Step 3.

1b. Read `handle_save_profile`

Open `app.py` and find `handle_save_profile`. It is the pattern every other handler follows.

```
def handle_save_profile(sport, experience, handedness, strengths, weaknesses, preferences):
    # 1. Turn raw form values into a profile dictionary (your Feature 2 code).
    profile = create_profile(sport, experience, handedness, strengths, weaknesses, preferences)

    # 2. Build the text we'll show the user.
    topics = get_coaching_topics(sport)
    message = f"Profile saved for **{sport}**.\n\n" + fallback_style_summary(profile)

    # 3. Return one value per output component, in the same order as `outputs=[...]`.
    return profile, message, gr.update(choices=topics, value=topics[0])
```

Now find where it is **wired up**, further down:

```
save_btn.click(
    handle_save_profile,
    inputs=[sport, experience, handedness, strengths, weaknesses, preferences],
    outputs=[profile_state, profile_msg, topics],
)
```

Read it aloud: *"When Save is clicked, call `handle_save_profile`. Give it the current values of these six components, **in this order**. Put the three things it returns into these three components, **in this order**."*

> **CONCEPT**
> Inputs and outputs are matched by **position**, not by name. The first thing in `inputs` becomes the first parameter; the first thing returned goes into the first component in `outputs`. If a function returns three values, `outputs` must list exactly three components.

1c. What is `gr.State`?

Near the bottom of `app.py` you will find these two lines:

```
profile_state = gr.State({})   # the athlete profile dictionary
chat_state = gr.State([])      # the conversation history (list of message dicts)
```

A `gr.State` holds an ordinary Python value that Gradio **remembers between clicks** for one browser tab. Each visitor gets their own copy. Look at the handler's return line again: the profile is the first thing returned, and `profile_state` is the first output, so that is how the new profile gets **saved**. If a handler changes the profile but does not return it into the state, the change is lost.

> **WHY**
> State lives in memory only. Refresh the page and it is gone. That is fine for this project.

1d. See how the other handlers use it

Skim `handle_style_fallback`, `handle_style_ai`, `handle_chat` and `handle_update_profile`. Each one: (1) takes the saved `profile` as an input, (2) checks `if not profile:` and shows the reminder `NEED_PROFILE` if the user skipped the first tab, (3) does its job, and (4) returns the profile again.

> **CHECKPOINT**
> Can you answer these? (1) What does `gr.State` do? (2) If a handler returns `profile, message, gr.update(...)`, which three components does `outputs` need to list, and in what order? (3) Which file talks to the AI? If yes, move on.

Step 2: Feature 2, create the athlete profile

File: `profile.py` **Function:** `create_profile(sport, experience, handedness, strengths, weaknesses, preferences)`

Goal: turn the six form values into one dictionary that the rest of the app passes around.

Ideas you need: dictionaries, `if / else`, lists, calling helper functions, `.strip()`.

Look at the target

The top of `profile.py` shows what a finished profile looks like. The values are different **types**: `sport` is a string, but `strengths` is a **list** of strings and `style_confirmed` is a **boolean** (`True` or `False`).

The starter version already builds the dictionary but leaves most of it empty, so the app runs:

```
profile = {
    "sport": sport,
    "experience": experience,
    "handedness": "N/A",
    "strengths": [],
    "weaknesses": [],
    "preferences": "",
    ...
}
```

Two helpers that are already written

- `side_matters(sport)` returns `True` for sports where left or right hand or foot can change advice (basketball, tennis, volleyball, soccer) and `False` for dance.
- `split_list("passing, speed , defense")` returns `["passing", "speed", "defense"]`. Gradio gives us **one string** from a textbox, but a list is easier to use in a prompt.

TODO 1: handedness, only when it matters

If `side_matters(sport)` is `True`, store the handedness value from the form. Otherwise store `"N/A"`. Why? The form's Dominant hand radio is hidden for dance, but it still has a value in the background, and we do not want to tell a dance coach "the athlete is left-handed".

```
if side_matters(sport):
    profile["handedness"] = handedness
else:
    profile["handedness"] = "N/A"
```

TODO 2: strengths and weaknesses as lists

Pass each text through `split_list` and store the **result** (the list) in the dictionary:

```
profile["strengths"] = split_list(strengths)
profile["weaknesses"] = split_list(weaknesses)
```

TODO 3: preferences

Store the text, with `.strip()` to remove extra spaces from both ends:

```
profile["preferences"] = preferences.strip()
```

Finished function

profile.py

```
def create_profile(sport, experience, handedness, strengths, weaknesses, preferences):
    """Build a brand-new profile dictionary from the form inputs.

    Parameters:
        sport (str): e.g. "tennis"
        experience (str): "Beginner", "Intermediate" or "Advanced"
        handedness (str): "Left", "Right", "Both" (may be ignored for some sports)
        strengths (str): comma-separated text, e.g. "serve, footwork"
        weaknesses (str): comma-separated text, e.g. "backhand"
        preferences (str): free text, e.g. "short drills, no equipment"

    Returns:
        dict: the profile (see the example at the top of this file).
    """
    # STARTER VERSION: only stores sport and experience, so the app runs.
    profile = {
        "sport": sport,
        "experience": experience,
        "handedness": "N/A",
        "strengths": [],
        "weaknesses": [],
        "preferences": "",
        "style_text": "",
        "style_summary": "",
        "style_confirmed": False,
    }

    if side_matters(sport):
        profile["handedness"] = handedness
    else:
        profile["handedness"] = "N/A"

    profile["strengths"] = split_list(strengths)
    profile["weaknesses"] = split_list(weaknesses)
    profile["preferences"] = preferences.strip()
    return profile
```

CHECKPOINT

Print the dictionary to check your work. In the terminal (with the venv on, inside this folder), type `python` to open an interactive Python session, then type the two lines after the `>>>` prompts. (Type `exit()` to leave.) You should see:

```
>>> from profile import create_profile
>>> print(create_profile('dance', 'Beginner', 'Left', 'turns', 'balance', '  slow music '))
{'sport': 'dance', 'experience': 'Beginner', 'handedness': 'N/A', 'strengths': ['turns'],
    'weaknesses': ['balance'], 'preferences': 'slow music', 'style_text': '', 'style_summary':
        '', 'style_confirmed': False}
```

Note `handedness` is `N/A` even though we passed `'Left'`, because dance is not a sport where side matters. Repeat with `'basketball'` and you should get `'Left'`. Then run the app, fill the form, and click **Save profile**: the summary should now show your strengths, weaknesses and preferences.

STUCK?

If the summary still says `not provided`, you probably wrote `split_list(strengths)` on its own line without storing the result. A function's answer disappears unless you assign it: `profile["strengths"] = split_list(strengths)`. `NameError: name 'split_list' ...` means a typo in the name. `IndentationError` means check the four-space indents.

Step 3: Feature 1, coaching topics for each sport

File: `coach.py` **Function:** `get_coaching_topics(sport)`

Goal: return a list of topics specific to the chosen sport. The Profile tab shows it in a dropdown.

Ideas you need: `if / elif / else`, list concatenation with `+`.

Read the example

Basketball is done for you:

```
if sport == "basketball":
    return ["Shooting form", "Ball handling", "Defense", "Court vision"] + GENERAL_TOPICS
```

`GENERAL_TOPICS` is a list defined above the function. Adding two lists with `+` glues them into one longer list. That way every sport gets its own four topics plus the three general ones (warm-up, focus, consistency).

TODO 1: add `elif` branches

Add one `elif` for each of soccer, volleyball, tennis and dance. Give each **3 to 4 topics specific to that sport**. Ask yourself: what would a real coach of that sport talk about? Tennis has the serve, the forehand, the backhand and footwork. Dance has technique, flexibility, rhythm and performance. Use your own ideas for the others. The sport names must match the spelling used in `SPORTS` at the top of `app.py` (lowercase).

TODO 2: keep the final `else`

Leave the last `else: return GENERAL_TOPICS` exactly as it is. If an unknown sport ever shows up, the app still gets a sensible list instead of crashing.

Finished function

`coach.py`

```
def get_coaching_topics(sport):
    """Return a list of coaching topics that make sense for the chosen sport.

    Parameters:
        sport (str): e.g. "basketball"

    Returns:
        list[str]: topic names shown in the UI dropdown.
    """
    # Basketball is done for you as an example of the pattern:
    if sport == "basketball":
        return ["Shooting form", "Ball handling", "Defense", "Court vision"] + GENERAL_TOPICS

    elif sport == "soccer":
        return ["Passing", "Dribbling", "Shooting", "Positioning"] + GENERAL_TOPICS
    elif sport == "volleyball":
        return ["Serving", "Passing", "Setting", "Attacking"] + GENERAL_TOPICS
    elif sport == "tennis":
        return ["Serve", "Forehand", "Backhand", "Footwork"] + GENERAL_TOPICS
    elif sport == "dance":
        return ["Technique", "Flexibility", "Rhythm", "Performance"] + GENERAL_TOPICS
    else:
        return GENERAL_TOPICS
```

Your topics can differ. That is fine and even good.

CHECKPOINT

In an interactive `python` session, type:

```
>>> from coach import get_coaching_topics
>>> print(get_coaching_topics('tennis'))
>>> print(get_coaching_topics('chess'))
['Serve', 'Forehand', 'Backhand', 'Footwork', 'Warm-up and recovery', 'Mental focus', 'Building
    consistency']
['Warm-up and recovery', 'Mental focus', 'Building consistency']
```

Then restart the app and change the **Sport** dropdown. The Coaching topics list should change to match each sport. Also notice the **Dominant hand / foot** radio disappears for dance. That comes from `side_matters`.

STUCK?

`SyntaxError` on an `elif` line usually means a missing colon `:`. If a sport shows only the three general topics, the name in your `elif` does not match (check for capital letters or a typo).

Step 4: Feature 3, describe your playing style

File: `profile.py` **Function:** `add_style_description(profile, style_text)`

Goal: save the athlete's own description of how they play into the profile.

Ideas you need: dictionary assignment, copying a dictionary, booleans.

Why `dict(profile)`?

Look at the first line of the function:

```
updated = dict(profile)
```

`dict(profile)` makes a **copy**. If you changed `profile` directly, you would be editing the very same object Gradio has stored, which can cause confusing bugs. The pattern in this project is: *copy, change the copy, return the copy.*

TODO 1: store the text

```
updated["style_text"] = style_text.strip()
```

TODO 2: reset `style_confirmed`

If the athlete edits their description, any summary they confirmed earlier might no longer be accurate. So set it back to `False` (the real boolean value, no quotes):

```
updated["style_confirmed"] = False
```

Finished function

profile.py

```
def add_style_description(profile, style_text):
    """Combine the user's free-text style description with the structured profile.

    Parameters:
        profile (dict): the current profile.
        style_text (str): the user's own words about how they play.

    Returns:
        dict: a copy of the profile that now includes the style text.
    """
    updated = dict(profile)
    updated["style_text"] = style_text.strip()
    updated["style_confirmed"] = False
    return updated
```

CHECKPOINT

Run the app. Save a profile on tab 1, go to tab 2, type a description such as `I like short passes` and click **Quick summary (no AI)**. The summary now ends with **In your own words: I like short passes**. Also try this in an interactive `python` session:

```
>>> from profile import create_profile, add_style_description
>>> p = create_profile('soccer', 'Beginner', 'Right', 'passing', '', '')
>>> p = add_style_description(p, '  I like short passes ')
>>> print(repr(p['style_text']), p['style_confirmed'])
'I like short passes' False
```

The extra spaces are gone, thanks to `.strip()`.

> **STUCK?**
> If you get `KeyError` or `NEED_PROFILE`, you skipped **Save profile** on tab 1. The handlers say "Please create your profile on the first tab" when the state is empty.

Step 5: Feature 5, the personalized system prompt

File: `coach.py` **Function:** `build_coaching_system_prompt(profile)`

Goal: write the instructions that tell the AI who it is and exactly who it is coaching. This is the most important function in the project. It is where "generic chatbot" becomes "your coach".

Ideas you need: `if / elif`, building a string with `+=`, f-strings, dictionary `.get()`.

System prompt vs. messages

Every request to the AI has two parts:

- The **system prompt**: standing instructions ("You are a coach. Here is the athlete.") The user never sees it.
- The **messages**: the conversation so far, from the user and the assistant.

We rebuild the system prompt from the profile **every time** the user sends a message. That means the moment a profile changes (Feature 7), the very next answer reflects it automatically.

What is already there

```
sport = profile.get("sport", "their sport")

prompt = (
    f"You are a friendly, knowledgeable {sport} coach.\n\n"
    f"ABOUT THE ATHLETE:\n{profile_to_text(profile)}\n\n"
)
```

`profile.get("sport", "their sport")` is the safe way to read a dictionary: if the key is missing you get the fallback instead of a crash. `profile_to_text` (already written) formats the profile as bullet lines. Try printing its output if you want to see what the AI "knows".

TODO 1: get the playing style

The athlete's own words are **already** inside `profile_to_text`. What we want to add is the **confirmed summary** once the athlete has approved one (Step 8). Read it from the profile, but only if it is confirmed:

```
style = ""
if profile.get("style_confirmed") and profile.get("style_summary"):
    style = profile["style_summary"]
```

Both conditions must be true, and an empty string counts as false, so this is also safe for a brand new profile.

TODO 2 and TODO 3: personalize with `if` / `elif`

Now add details to the prompt one at a time with `prompt += ...`. Each `if` checks the profile and adds one sentence:

- If `style` is not empty, add the confirmed summary.
- If handedness is `"Left"`: *"The athlete is left-handed. Do not assume right-handed technique; mirror any right-handed examples."*
- `elif` handedness is `"Both"`: say both sides can be trained.
- If experience is `"Beginner"`: ask for simple language and basics first.
- If the preferences text is not empty, tell the model to respect it.

```
handedness = profile.get("handedness")
if handedness == "Left":
    prompt += ("The athlete is left-handed. Do not assume right-handed technique; "
               "mirror any right-handed examples.\n")
elif handedness == "Both":
    prompt += "The athlete can train both sides, so include drills for each side.\n"
```

Notice that for `"Right"` (or `"N/A"`) **nothing** is added. That is fine: the model assumes the usual.

TODO 4: the safety rule

Coaches must not just do whatever they are told. Add a rule that the coach should respect preferences, but if a technique could be unsafe or ineffective, explain why and suggest a safer alternative. Also tell it not to make claims about professional athletes it is unsure of.

TODO 5: the style rule

Add a short rule: *"Keep answers concise and practical."*

Finished function

coach.py

```
def build_coaching_system_prompt(profile):
    """Build the "system prompt" that tells the LLM who it is and who it's coaching.

    The system prompt is sent with EVERY request. Because we rebuild it from the
    profile each time, profile updates (Feature 7) automatically affect the advice.

    Parameters:
        profile (dict): the athlete profile.

    Returns:
        str: the system prompt.
    """
    sport = profile.get("sport", "their sport")
    # The athlete's own words are already inside profile_to_text(). Once they have
    # confirmed an AI summary (Feature 4), we add that too.
    style = ""
    if profile.get("style_confirmed") and profile.get("style_summary"):
        style = profile["style_summary"]

    prompt = (
        f"You are a friendly, knowledgeable {sport} coach.\n\n"
        f"ABOUT THE ATHLETE:\n{profile_to_text(profile)}\n\n"
    )

    if style:
        prompt += f"CONFIRMED SUMMARY OF THEIR PLAYING STYLE:\n{style}\n\n"

    handedness = profile.get("handedness")
    if handedness == "Left":
        prompt += ("The athlete is left-handed. Do not assume right-handed technique; "
                   "mirror any right-handed examples.\n")
    elif handedness == "Both":
        prompt += "The athlete can train both sides, so include drills for each side.\n"

    if profile.get("experience") == "Beginner":
        prompt += "The athlete is a beginner: use simple language and cover the basics first.\n"

    if profile.get("preferences"):
        prompt += f"Respect these preferences: {profile['preferences']}.\n"
```

```
    prompt += (
        "\nRespect the athlete's preferences, but if a technique could be unsafe or "
        "ineffective, say so clearly, explain why, and suggest a safer alternative. "
        "Do not make claims about professional athletes or performance results you "
        "are not sure about.\n"
    )
    prompt += "Keep answers concise and practical.\n"
    return prompt
```

Read it as if you were the AI

In an interactive `python` session, run these lines and **read the output slowly**. It is everything the AI will know about the athlete.

```
>>> from profile import create_profile
>>> from coach import build_coaching_system_prompt
>>> me = create_profile('basketball', 'Beginner', 'Left', 'passing, speed', 'free throws', 'short
    drills')
>>> print(build_coaching_system_prompt(me))
```

You should see:

```
You are a friendly, knowledgeable basketball coach.

ABOUT THE ATHLETE:
- Sport: basketball
- Experience level: Beginner
- Handedness: Left
- Strengths: passing, speed
- Weaknesses: free throws
- Preferences: short drills
- Playing style (user's words): none given

The athlete is left-handed. Do not assume right-handed technique; mirror any right-handed
    examples.
The athlete is a beginner: use simple language and cover the basics first.
Respect these preferences: short drills.

Respect the athlete's preferences, but if a technique could be unsafe or ineffective, say so
    clearly, ...
Keep answers concise and practical.
```

CHECKPOINT

Compare two athletes. Build a second athlete with `'Right'` and `'Advanced'` instead of `'Left'` and `'Beginner'`, and print its prompt. The "left-handed" and "beginner" lines must **disappear**. If the two prompts are identical, your `if` statements are not working. This "compare two profiles" test is the best way to check that a profile really reaches the AI.

STUCK?

If nothing is added for left-handed athletes, check spelling: the form gives `"Left"` with a capital L, and `==` is case-sensitive. If you get a `KeyError`, use `profile.get("handedness")` rather than `profile["handedness"]`. If the text runs together without line breaks, add `\n` at the end of each sentence.

Step 6: Feature 6, trace the chat, then talk to your coach

Files: `app.py` (`handle_chat`), `coach.py` (`get_coaching_response`), `ai_helper.py` (`ask_llm`)

Goal: there is almost nothing to write here. The `TODO` in `handle_chat` says: *"Nothing to change here, but trace this call!"* Following a request through three files is a real programmer skill. This step teaches you how to do it, and how the AI gets its memory.

Trace 1: `handle_chat` in `app.py`

```
reply = get_coaching_response(message, chat_history, profile)

chat_history = chat_history + [
    {"role": "user", "content": message},
    {"role": "assistant", "content": reply},
]
return "", chat_history, chat_history
```

- `message` is what the user typed. `chat_history` is the earlier conversation, a list of dictionaries, each with a `"role"` (`"user"` or `"assistant"`) and `"content"` (the text). It comes out of `chat_state` .
- The handler asks `get_coaching_response` for the reply, then **adds both the user's message and the reply** to the list.
- It returns **three** values: an empty string (which clears the textbox), the history (which updates the visible chat window), and the history again (which saves it into `chat_state`).

Trace 2: `get_coaching_response` in `coach.py`

```
if not has_api_key():
    return demo_response(message, profile)

system_prompt = build_coaching_system_prompt(profile)          # who the AI is + athlete info
messages = list(history) + [{"role": "user", "content": message}]  # past + new message
return ask_llm(system_prompt, messages)
```

Two paths: no key means the canned `demo_response` . With a key, it builds the **system prompt** (your Step 5 work) and a **messages** list that is the **entire past conversation plus the new message**.

> **WHY**
> The AI has **no memory of its own**. Every request starts from nothing. If we did not resend the history, the follow-up "What about my left hand?" would be meaningless: "my left hand" compared to what? Resending the history on every message is the only reason the coach seems to remember.

Optional TODO: a friendly reminder for an empty profile

The comment at the top of `get_coaching_response` suggests: if the profile is empty, return a reminder. Add this before the `has_api_key` check:

```
if not profile:
    return "Please create your profile on the first tab so I can coach you personally."
```

`not profile` is `True` for an empty dictionary `{}` , which is what `handle_chat` passes when the user skipped the first tab.

Trace 3: `ask_llm` in `ai_helper.py` (read only)

This file is finished, so just read it. The key parts:

1. `has_api_key()` returns `True` only if `GEMINI_API_KEY` is set (it is read from `.env` by `load_dotenv()`).
2. It trims each message down to its role and text, and renames the role `assistant` to `model` (that is what Gemini calls the AI's side), because Gradio may attach extra fields the API does not accept.
3. `client.models.generate_content(model=..., contents=contents, config=...)` is the actual call to the AI. Notice the system prompt (`system_instruction` , set in the `config`) and the messages (`contents`) are **separate** arguments, exactly as described in Step 5.
4. A `try / except` turns any failure into a readable message instead of a crash.

Finished function (empty-profile reminder included)

coach.py

```
def get_coaching_response(message, history, profile):
    """Produce the coach's reply to the user's newest message.

    Parameters:
        message (str): what the user just typed.
        history (list[dict]): earlier messages, each {"role": ..., "content": ...}.
            Sending history lets the AI remember what was already said, so
            "What about my left hand?" makes sense as a follow-up.
        profile (dict): the athlete profile.

    Returns:
        str: the coach's reply.
    """
    if not profile:
        return "Please create your profile on the first tab so I can coach you personally."

    if not has_api_key():
        return demo_response(message, profile)

    system_prompt = build_coaching_system_prompt(profile)           # who the AI is + athlete info
    messages = list(history) + [{"role": "user", "content": message}]  # past + new message
    return ask_llm(system_prompt, messages)
```

See exactly what the AI would receive

You can swap the AI for a fake one that just **prints** what it was sent. Create a new file called `peek.py` next to `app.py` :

```
import coach
from profile import create_profile

def fake_ask_llm(system_prompt, messages, max_tokens=800):
    print("SYSTEM PROMPT:")
    print(system_prompt)
    print("MESSAGES SENT:")
    for m in messages:
        print("  ", m["role"], "->", m["content"])
    return "(fake reply)"

coach.ask_llm = fake_ask_llm       # swap the real LLM for our fake
coach.has_api_key = lambda: True   # pretend a key exists

me = create_profile("basketball", "Beginner", "Left", "passing, speed", "free throws", "short
    drills")
history = [
    {"role": "user", "content": "How do I improve my shot?"},
    {"role": "assistant", "content": "Start close to the hoop and bend your knees."},
]
coach.get_coaching_response("What about my left hand?", history, me)
```

Run `python peek.py`. After the system prompt from Step 5 you will see:

```
MESSAGES SENT:
   user -> How do I improve my shot?
   assistant -> Start close to the hoop and bend your knees.
   user -> What about my left hand?
```

That list is the AI's entire "memory". Three messages, and the newest one is last.

> **CONCEPT**
> This trick (replacing a function with a fake one to see what it gets) is called **mocking**, and professional programmers use it all the time to test code without calling a paid service.

Now try it for real

1. Get a Gemini API key from your teacher (or create one at aistudio.google.com/apikey).
2. Open `.env` and set the key with no quotes and no spaces:

```
GEMINI_API_KEY=your-key-here
LLM_MODEL=gemini-3.5-flash
```

3. Stop and restart the app. The **demo mode** banner at the top should be gone.
4. Save a profile, go to the Chat tab, and ask: *How can I improve my shooting?* Then ask a follow-up like *What about my left hand?* The coach should understand the follow-up. That is the history working.

Test the personalization

- **Two profiles.** Save a left-handed beginner, ask a question, and write down the answer. Refresh the page (this clears the state), save a right-handed advanced athlete, and ask the same question. If the answers are almost identical, the profile is not reaching the prompt. Go back to Step 5.
- **An unsafe request.** Ask something like *"I want to practice with no warm-up and train through pain."* The coach should explain the risk and suggest a safer way instead of just complying. That is your safety rule working.

CHECKPOINT

Without a key the demo reply should mention your sport and handedness. With a key, a follow-up question must make sense. If you see `The AI request failed: ...`, read the end of the message: `API key not valid` or `403` means a wrong or misspelled key, and a `model` error means `LLM_MODEL` has a typo. The key file must be named exactly `.env` and sit next to `app.py`.

STUCK?

If the banner still shows after adding a key, you did not restart the app, or the `.env` is in the wrong folder. If the chat window shows an error about messages, make sure you did not change the two-key dictionaries in `handle_chat` (`"role"` and `"content"`).

Step 7: Feature 4, the AI playing-style analysis prompt

File: `coach.py` **Function:** `build_style_analysis_prompt(profile, include_pro_comparisons=False)`

Goal: ask the AI to summarize the athlete's playing style, **without inventing facts**, and let the user turn pro-athlete comparisons on or off.

Ideas you need: string building, `if / else`, a boolean parameter, prompt-writing for safety.

The starter

```
prompt = (
    "Summarize this athlete's playing style in 3–5 sentences.\n\n"
    f"ATHLETE PROFILE:\n{profile_to_text(profile)}\n"
)
```

The caller is `analyze_style`, which is already written: with no key it returns the plain fallback summary; with a key it sends your prompt to the AI with the system prompt `"You are a careful sports coach who never invents facts."`

TODO 1: rules against inventing facts

Language models sometimes fill gaps with confident-sounding guesses. Tell it not to:

```
prompt += (
    "\nRULES:\n"
    "- Only use the information given above. If something is unknown, say so.\n"
    "- Do not invent facts about the athlete.\n"
)
```

TODO 2: two possibility lists

Ask for "Possible strengths" and "Possible tendencies", and tell the model to use hedging words ("may", "seems"). Because the AI is guessing from a few sentences, the answer should *sound* like a guess:

```
prompt += (
    "\nAfter the summary, add two short lists:\n"
    "Possible strengths:\n"
    "Possible tendencies:\n"
    'Word every item as a possibility, using words like "may" or "seems".\n'
)
```

The single quotes on the last line let us put double quotes inside the string.

TODO 3: ask the athlete to confirm

```
prompt += "\nEnd by asking the athlete to confirm or correct this summary.\n"
```

This sets up the **Confirm summary** button you will finish in Step 8.

TODO 4: pro comparisons, on or off

The checkbox in the app passes `include_pro_comparisons` as `True` or `False`. Use `if` / `else`. When `True`, add a section titled exactly `Illustrative comparisons (NOT verified matches)` asking for one or two well-known athletes with a similar **described** style, and tell the model not to claim statistics it is unsure about. When `False`, tell it to skip comparisons. Finally, return the prompt.

> **CONCEPT**
> Why the label "NOT verified matches"? Language models can state wrong facts confidently. Labeling the section keeps the user from treating a guess as truth. Good apps are honest about what the AI can and cannot know.

Finished function

coach.py

```
def build_style_analysis_prompt(profile, include_pro_comparisons=False):
    """Build the prompt asking the LLM to summarize the user's playing style.

    Parameters:
        profile (dict): the athlete profile (including "style_text").
        include_pro_comparisons (bool): if True, ask for OPTIONAL illustrative
            comparisons to professional athletes.

    Returns:
        str: the prompt to send to the LLM.
    """
    prompt = (
        "Summarize this athlete's playing style in 3-5 sentences.\n\n"
        f"ATHLETE PROFILE:\n{profile_to_text(profile)}\n"
    )

    prompt += (
        "\nRULES:\n"
        "- Only use the information given above. If something is unknown, say so.\n"
        "- Do not invent facts about the athlete.\n"
    )

    prompt += (
        "\nAfter the summary, add two short lists:\n"
        "Possible strengths:\n"
        "Possible tendencies:\n"
        'Word every item as a possibility, using words like "may" or "seems".\n'
    )

    prompt += "\nEnd by asking the athlete to confirm or correct this summary.\n"

    if include_pro_comparisons:
        prompt += (
            "\nAlso add a section titled "
            '"Illustrative comparisons (NOT verified matches)". '
            "Name 1-2 well-known athletes with a similar DESCRIBED style. "
            "Do not claim statistics or facts you are not sure about.\n"
        )
    else:
        prompt += "\nDo not compare the athlete to any professional athletes.\n"
    return prompt
```

Check both branches

Test it in an interactive `python` session. The pro section must appear **only** when the flag is `True`:

```
>>> from profile import create_profile, add_style_description
>>> from coach import build_style_analysis_prompt
>>> p = create_profile('basketball', 'Beginner', 'Left', 'passing', 'free throws', '')
>>> p = add_style_description(p, 'I am patient')
>>> print('Illustrative' in build_style_analysis_prompt(p, False))
False
>>> print('Illustrative' in build_style_analysis_prompt(p, True))
True
```

Then run `print(build_style_analysis_prompt(p, True))` to read the whole prompt. With `True` it ends like this:

```
End by asking the athlete to confirm or correct this summary.

Also add a section titled "Illustrative comparisons (NOT verified matches)". Name 1-2 well-known
    athletes with a similar DESCRIBED style. Do not claim statistics or facts you are not sure
        about.
```

CHECKPOINT

With a key, go to the **Playing style** tab, type a description, tick or untick the pro-comparison box, and click **AI style analysis**. The summary should only use details you gave it, use words like "may" and "seems", and end by asking you to confirm. If it invents things (a position or a favorite team you never mentioned), make the RULES stronger and try again. Without a key you will see the demo banner plus the plain summary, which is expected.

STUCK?

If the pro section appears even when the box is unticked, your `if` is probably checking a string instead of the boolean. Write `if include_pro_comparisons:` with no `== "True"` . If you forgot `return prompt` , the AI gets `None` .

Step 8: Feature 7, update the profile and confirm the summary

Files: `profile.py` (`update_profile`) and `app.py` (`handle_confirm_style`)

Goal: change one field of an existing profile safely, then use it to save the user's confirmed style summary. We do `update_profile` first because `handle_confirm_style` depends on it.

Ideas you need: `if field in dictionary` , `elif` , copying, lists vs strings vs booleans.

Part A: `update_profile(profile, field, new_value)`

The Update tab gives us a field name and a new value typed as text. The trick is that different fields hold different **types**:

Field	Type it holds	What to do with the text
`strengths` , `weaknesses`	list	`split_list(new_value)`
`style_confirmed`	boolean	store `True` or `False` , never a string
everything else	string	store it as is

The function starts with `updated = dict(profile)` , a copy (as in Step 4).

TODO 3 first: ignore unknown fields

Everything should be inside an `if field in updated:` check. If someone asks to update a field that does not exist (for example `"colour"`), nothing happens and the profile comes back unchanged. Without the check you would silently create junk keys.

TODO 1: the two list fields

```
if field in ("strengths", "weaknesses"):
    updated[field] = split_list(new_value)
```

A string like `"defense, hustle"` becomes the list `["defense", "hustle"]` .

TODO 2: everything else, and the boolean

Add `elif` and `else` branches. The scaffold's hint says it stores strings, so booleans need special care. A string `"False"` is **truthy** in Python (any non-empty string is), so storing it as text would make the profile look confirmed. The scaffold lets you extend the function, so we add one extra branch that only accepts a real `True` :

```
elif field == "style_confirmed":
    updated[field] = new_value is True  # a real True/False, never a string
else:
    updated[field] = new_value
```

Finished function

profile.py

```
def update_profile(profile, field, new_value):
    """Change ONE field of an existing profile and return the updated profile.

    Parameters:
        profile (dict): the current profile.
        field (str): which key to change, e.g. "preferences" or "strengths".
        new_value (str): the new value typed by the user.

    Returns:
        dict: the updated profile (a copy, so the original is untouched).
    """
    updated = dict(profile)  # copy, so we don't accidentally change the original

    if field in updated:
        if field in ("strengths", "weaknesses"):
            updated[field] = split_list(new_value)
        elif field == "style_confirmed":
            updated[field] = new_value is True  # a real True/False, never a string
        else:
            updated[field] = new_value
    return updated
```

CHECKPOINT

In an interactive `python` session, run:

```
>>> from profile import create_profile, update_profile
>>> p = create_profile('basketball', 'Beginner', 'Left', 'passing', '', '')
>>> q = update_profile(p, 'strengths', 'defense, hustle')
>>> print(q['strengths'], p['strengths'])
['defense', 'hustle'] ['passing']
>>> print(update_profile(p, 'colour', 'blue') == p)
True
>>> print(update_profile(p, 'style_confirmed', True)['style_confirmed'])
True
```

The first output line shows the new list **and** that the original `p` was not touched (the copy worked). The second shows the unknown field was ignored. The third shows the boolean was stored. In the app, go to **Update profile**, pick `preferences`, type a new value and click **Update**; you should see it in the updated summary. Then chat again: the next reply reflects the change, because the system prompt is rebuilt every time.

NOTE

Look at `EDITABLE_FIELDS` at the top of `app.py`. It contains only `preferences, strengths, weaknesses, experience, handedness`. `style_confirmed` is deliberately **not** in the dropdown, so a user typing text into the Update tab can never break the boolean. Only our own code can set it.

Part B: `handle_confirm_style(profile, edited_summary)`

What it does: on the Playing style tab, the AI summary also fills an editable box. The user can fix anything the AI got wrong, then click **Confirm summary**. Your handler saves it.

TODO 1: save the summary

Use `update_profile` to put the (edited) text into `"style_summary"`.

TODO 2: mark it confirmed

Call `update_profile` again to set `"style_confirmed"` to `True`. Because of the branch you just wrote, this stores a real boolean.

We also added a small guard: if the box is empty (nothing to confirm), say so and leave the profile alone.

app.py

```
def handle_confirm_style(profile, edited_summary):
    """Feature 4: the user confirms (or corrects, by editing) the style summary."""
    if not profile:
        return profile, NEED_PROFILE
    if not edited_summary.strip():
        return profile, "Nothing to confirm yet. Run a style analysis first."
    profile = update_profile(profile, "style_summary", edited_summary.strip())
    profile = update_profile(profile, "style_confirmed", True)
    return profile, "Summary confirmed. Your coach will use it in every answer."
```

CHECKPOINT

Save a profile, type a style description, click **Quick summary (no AI)** (this is the only one that works with no key), copy some text into the "Confirm or correct" box and click **Confirm summary**. You should see `Summary confirmed. Your coach will use it in every answer.` Clicking Confirm with an empty box should instead say `Nothing to confirm yet. Run a style analysis first.`

Close the loop

Remember Step 5, TODO 1? `build_coaching_system_prompt` adds the summary to the system prompt **only** when `style_confirmed` is `True` and there is a `style_summary`. Now that both are set, the coach has a new section:

```
CONFIRMED SUMMARY OF THEIR PLAYING STYLE:
Patient playmaker who tends to drive right.
```

Also, if the user edits their description again in Step 4's `add_style_description`, `style_confirmed` goes back to `False`, so a stale summary stops being used. All seven features now work together.

STUCK?

If Confirm always says `TODO: confirmation is not saved yet`, you are editing the wrong file or did not restart. If the confirmation does not stick, check that you reassign: `profile = update_profile(profile, ...)` (the function returns a **copy**; it does not change the original), and that you return `profile` as the first value so Gradio saves it in the state.

Step 9: Optional extensions

Finished early? Each of these is a real feature. Do them in any order.

1. **Save and load profiles.** Use `json.dump(profile, file)` and `json.load(file)` to save a profile to `profile.json` and read it back. Add a Download and an Upload button in `app.py`. (The profile only contains strings, lists and booleans, so JSON handles it perfectly.)
2. **Goal tracking.** Add a `goals` list to the profile in `create_profile`, a way to mark goals done, and a line in `profile_to_text` and the system prompt such as "Goals completed: 2 of 4". Think about which function needs to change for each piece.
3. **Training plans.** Add a "Generate a weekly plan" button that builds a prompt from the profile **and** the topic chosen in the Coaching topics dropdown (you will need to pass that dropdown as an extra input). Reuse the ideas from Steps 5 and 7.
4. **Streaming replies.** Make the text appear as it is written. The Gemini library has `client.models.generate_content_stream(...)`, and Gradio handlers can `yield` partial results. This one touches `ai_helper.py`.
5. **A friendlier empty-state.** Show a short "what can I ask?" list in the chat using the coaching topics for the profile's sport.
6. **Swap the AI provider.** The app was designed so only `ai_helper.py` knows which AI it talks to. Change just that file to use a different provider (for example Claude or OpenAI) and see that nothing else breaks.

What you learned

- How a **profile dictionary becomes a prompt**, and why that is how an AI can "know" a user.
- The Gradio pattern: **event, function, inputs, outputs**, and that `gr.State` remembers Python values between clicks.
- **System prompt vs. messages**, and why the whole chat history is resent every time.
- **Dictionaries**, safe lookups with `.get()`, copying with `dict(...)`, and checking `if key in dictionary`.
- `if / elif / else`, list concatenation, and building long strings with `+=` and f-strings.
- Handling **different data types** (string, list, boolean) and why `"False"` is not `False`.
- **Prompt safety**: rules against inventing facts, labelled guesses, and a safety rule for unsafe requests.
- How to **trace** code across files, and how to **mock** a function to see what your program really sends.